Big Data Analytics: Group Project Proposal

ChronoStream-RAG: Testing AI Answers When Document Updates Arrive Out of Order

Lynette Osako Student ID: 8000290404
Panhapiseth Lim Student ID: 8000265274
Rajan Yadav Student ID: 8000277867

Abstract

Some AI systems search documents before generating answers. When those documents change, updates may arrive late, repeat, or finish processing out of order. We will test whether these conditions cause a system to use an older document version after it has received a newer one. Our project will use selected Wikimedia document histories to build and test a system that processes updates as they arrive.

We will send the same updates through the system under controlled delays, repeated deliveries, and interruptions. For each update, we will record its source version, when it arrives, and when it becomes available for answering questions. This will distinguish information that has not arrived from errors in processing information the system already holds. We will compare four methods: applying updates in arrival order, checking version numbers before replacing documents, keeping the full document history, and keeping a limited working history while correcting records affected by late updates.

All methods will use the same documents, questions, document search model, and language model. We will check whether the system selects the appropriate document versions and whether its answers reflect their content. This evaluation will measure agreement with the received documents, without assuming that every statement in those documents is true. We will also measure processing speed, memory use, update delays, and recovery time after interruptions. The goal is to determine which checks prevent incorrect answers and when simpler methods are sufficient. The project will produce a working system, an interactive dashboard, documented code, and a report explaining the results and limitations.

Project objectives

The project aims to explain when delayed or repeated updates cause incorrect answers and which checks prevent them. We will compare the methods by answer accuracy, processing speed, memory use, and recovery time.

Workflow and tools

Kafka will receive the updates, PySpark will process them, and Parquet files will preserve the document history. Qdrant will support document search, and a small language model will generate answers. A Python dashboard will display answers, source versions, and processing times. We will begin with about 1 GB of data and aim to test 5 to 10 GB, reporting original content separately from repeated records.

Tentative timeline

During weeks 1 and 2, we will select the data, set up the tools, and test basic version checks. Weeks 3 to 5 will focus on receiving and resending updates under controlled conditions. Weeks 6 to 8 will cover the remaining methods and answer tests. In weeks 9 and 10, we will connect the system to the dashboard and measure performance. Weeks 11 and 12 will cover dashboard completion, testing under heavier workloads, the report, and presentation.